

Artificial Intelligence

Complete Question & Answer Study Guide

5 Questions $\times$ 10 Marks = 50 Marks — A self-contained guide with full diagrams, worked solutions, and intuition.

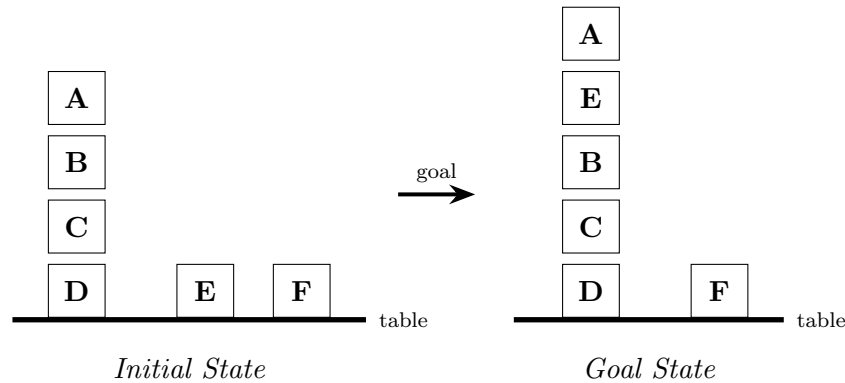
Contents

1	Question 1 — Blocks World: Steepest-Ascent Hill Climbing	2
2	Question 2 — TSP as State-Space Search with A*	7
3	Question 3 — A* Algorithm: Expanding Node N	10
4	Question 4 — Minimax with Alpha-Beta Pruning on Tic-Tac-Toe	13
5	Question 5 — Intelligent Agent for Smart Farm Management	18

Question 1 — Blocks World: Steepest-Ascent Hill Climbing

The Question

Consider the Blocks World problem below. A block may be moved onto another block or onto the table. Two heuristic functions guide a **steepest-ascent hill-climbing** search.



Heuristic functions:

- $h_1(n)$ — **Individual Block Heuristic**. Each block scores +1 if it rests on its correct support (per the goal), -1 otherwise:

$$h_1(n) = val_A + val_B + val_C + val_D + val_E + val_F, \quad val_X \in \{+1, -1\}$$

- $h_2(n)$ — **Entire-Pile Heuristic**. Examine every pile as a unit. If the pile's configuration is entirely correct up to the goal state, every block in it scores +1. If *any* block in the pile is wrong, *every* block in that pile scores -1.

Task: Generate successors from the initial state; evaluate h_1 and h_2 for each; identify which move steepest-ascent picks under each heuristic; trace toward the goal; critically compare both heuristics.

Background Concepts

Steepest-Ascent Hill Climbing

A *local search* that, at every step, evaluates all neighbours and jumps to the one with the highest heuristic value. It never backtracks and keeps no search history.

What can go wrong?

- **Local maximum:** a state better than all neighbours but not the goal — search halts permanently.
- **Plateau:** all neighbours score equally — no clear direction.

Blocks World correctness rule

Block X is *correct* iff the block *immediately below* X in the current state is exactly what the goal specifies should be below X . h_2 extends this: if even one block in a pile is wrong, every block in that pile is penalised.

Goal, read bottom-to-top

D on table $\rightarrow$ C on D $\rightarrow$ B on C $\rightarrow$ E on B $\rightarrow$ A on E. Plus F alone on the table.

Step-by-Step Solution**Step 1 — Evaluate the initial state**

h_1 (initial):

Block	Currently rests on	Goal: should rest on	Score
D	Table	Table	+1
C	D	D	+1
B	C	C	+1
E	Table	B	-1
A	B	E	-1
F	Table	Table	+1
$h_1(\text{initial})$			+3

h_2 (initial): Three piles exist: $[D, C, B, A]$ (bottom to top), $[E]$, $[F]$.

Pile	Verdict	Score
$[D, C, B, A]$	D, C, B are in correct relative order. But A sits on B; the goal requires A to sit on E. One wrong block $\Rightarrow$ the <i>entire</i> pile is penalised.	$4 \times (-1) = -4$
$[E]$	E is on the table; the goal requires E to be on B. Wrong.	-1
$[F]$	F is on the table; the goal agrees. Correct.	+1
$h_2(\text{initial})$		-4

Key Insight: $h_1 = +3$ rewards D, C, B, F individually. $h_2 = -4$ penalises the entire big pile because A is wrong. This “philosophy” gap matters greatly later.

Step 2 — Enumerate successors

Only the *topmost* block of each pile can move. Moveable blocks: **A** (top of the 4-stack), **E** (singleton), **F** (singleton).

Label	Move
S1	Move A onto E
S2	Move A onto F
S3	Move A to table (A becomes its own singleton)
S4	Move E onto A
S5	Move E onto F
S6	Move F onto A
S7	Move F onto E

Step 3 — Score every successor

Goal reference: D/table ✓, C/D ✓, B/C ✓, E/B ✓, A/E ✓, F/table ✓.

S1 — Move A onto E. Piles: $[B, C, D]$, $[A, E]$, $[F]$.

- h_1 : D ✓+1; C ✓+1; B ✓+1; E/table $\times -1$ (E should be on B); A/E ✓+1; F ✓+1.
 $h_1 = +4$
- h_2 : $[D, C, B]$ all correct = +3. $[E, A]$: E on table is wrong $\Rightarrow$ entire pile = -2. $[F]$ ✓ = +1. $h_2 = +2$

S2 — Move A onto F. Piles: $[B, C, D]$, $[E]$, $[A, F]$.

- h_1 : D ✓; C ✓; B ✓; E/table $\times$; A/F $\times$ (A should be on E); F/table ✓. $h_1 = +2$
- h_2 : $[D, C, B] = +3$. $[E]$ wrong = -1. $[F, A]$: A on F is wrong = -2. $h_2 = 0$

S3 — Move A to table. Piles: $[B, C, D]$, $[A]$, $[E]$, $[F]$.

- h_1 : D ✓; C ✓; B ✓; E/table $\times$; A/table $\times$; F ✓. $h_1 = +2$
- h_2 : $[D, C, B] = +3$. $[A]$ wrong = -1. $[E]$ wrong = -1. $[F]$ ✓ = +1. $h_2 = +2$

S4 — Move E onto A. Piles: $[E, A, B, C, D]$ (bottom: D,C,B,A,E), $[F]$.

- h_1 : D ✓; C ✓; B ✓; A/B $\times$; E/A $\times$; F ✓. $h_1 = +2$
- h_2 : $[D, C, B, A, E]$: A on B is wrong $\Rightarrow$ all 5 blocks penalised = -5. $[F]$ ✓ = +1.
 $h_2 = -4$

S5 — Move E onto F. Piles: $[A, B, C, D]$, $[E, F]$.

- h_1 : D ✓; C ✓; B ✓; A/B $\times$; E/F $\times$; F/table ✓. $h_1 = +2$
- h_2 : $[D, C, B, A]$: A wrong = -4. $[F, E]$: E on F wrong = -2. $h_2 = -6$

S6 — Move F onto A. Piles: $[F, A, B, C, D]$, $[E]$.

- h_1 : D ✓; C ✓; B ✓; A/B $\times$; E/table $\times$; F/A $\times$. $h_1 = 0$
- h_2 : $[D, C, B, A, F]$: A wrong = -5. $[E]$ wrong = -1. $h_2 = -6$

S7 — Move F onto E. Piles: $[A, B, C, D]$, $[F, E]$.

- h_1 : D ✓; C ✓; B ✓; A/B $\times$; E/table $\times$; F/E $\times$. $h_1 = 0$

- h_2 : $[D, C, B, A] = -4$. $[E, F] = -2$. $h_2 = -6$

Step 4 — Summary and steepest-ascent selection

State	Move	h_1	h_2
Initial	—	+3	-4
S1	A onto E	+4	+2
S2	A onto F	+2	0
S3	A to table	+2	+2
S4	E onto A	+2	-4
S5	E onto F	+2	-6
S6	F onto A	0	-6
S7	F onto E	0	-6

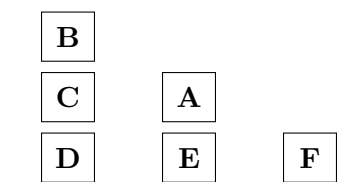
- Under h_1 : Select **S1** — score +4 is uniquely highest.
- Under h_2 : S1 and S3 *tie* at +2. Tie-breaking (first in list) selects S1 or S3.

Step 5 — What happens after S1?

Under h_1 — **trapped at a local maximum**:

After S1, piles are $[B, C, D]$, $[A, E]$, $[F]$. Moveable tops: B, A, F. Every possible next move scores $h_1 = +2$, which is *below* the current +4. Hill climbing refuses to move — it is stuck.

The goal is *not* reached. This is a **local maximum**.



State S1: $h_1 = +4$, hill climbing halts here

To finish, A must be lifted off E — but that drops h_1 back to +2. Hill climbing refuses.

Key Insight: A is “correctly” on E, earning +1. But the whole assembly $[A, E]$ needs to be placed on B. To do that, A must temporarily leave E — making h_1 drop. Hill climbing cannot see past this immediate loss, so it treats S1 as the best it can do.

Under h_2 — **successful via S3**:

From S3: piles $[B, C, D]$, $[A]$, $[E]$, $[F]$, $h_2 = +2$.

Move E onto B. New piles: $[E, B, C, D]$, $[A]$, $[F]$.

- $[D, C, B, E]$: D on table ✓, C on D ✓, B on C ✓, E on B ✓ $\Rightarrow$ all +4.
- $[A]$: wrong = -1. $[F]$ ✓ = +1. Total = +4. Improvement!

Move A onto E. New piles: $[A, E, B, C, D], [F]$.

- $[D, C, B, E, A]$: all five blocks in goal order ✓ = +5. $[F]$ ✓ = +1. Total = +6. **Goal!**

Step 6 — Full solution path

#	Action	h_1	h_2
0	Initial $[A, B, C, D], [E], [F]$	+3	-4
1	Move A to table $\rightarrow [B, C, D], [A], [E], [F]$	+2	+2
2	Move E onto B $\rightarrow [E, B, C, D], [A], [F]$	—	+4
3	Move A onto E $\rightarrow [A, E, B, C, D], [F]$	—	+6
GOAL in 3 moves		+6	

Step 7 — Critical comparison

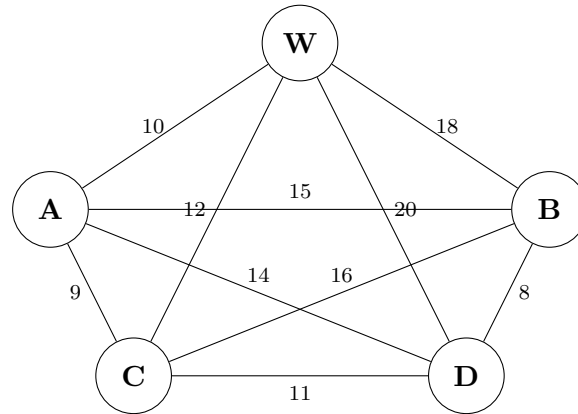
Criterion	h_1 — Individual Block	h_2 — Entire Pile
Granularity	Each block scored in isolation.	Each block scored in context of its pile.
Guidance	Moderate; can be misled by partial wins.	Stronger; punishes tainted piles.
Local maxima	Prone — gets stuck at S1.	Less prone — correctly devalues S1.
Optimality	No guarantee (hill climbing is incomplete).	No guarantee (same algorithm limitation).
Verdict	Simpler but misleads here.	More informative; finds goal; still fallible.

Key Insight: The core limitation is the **algorithm**, not just the heuristic. Hill climbing never backtracks. To guarantee a solution, use A* (with admissible heuristic), which is complete and optimal. To escape local maxima probabilistically, use random restarts or simulated annealing.

Question 2 — TSP as State-Space Search with A*

The Question

A vehicle starts at Warehouse W, visits relief centres A, B, C, D exactly once each, and returns to W. Find the minimum-cost route. The graph with edge costs is:



Constraints: Start at W; visit A, B, C, D exactly once; return to W; minimise total edge cost.

Background Concepts

This is the Travelling Salesman Problem (TSP)

Find the shortest Hamiltonian cycle. With 4 cities, there are $4! = 24$ possible orderings — small enough to enumerate, but we use A* to demonstrate principled pruning.

Why greedy (cheapest next edge) fails

A locally cheap first step can trap you into expensive later steps. Minimum total cost requires considering the entire route.

A* for TSP

$$f(n) = g(n) + h(n):$$

- $g(n)$ = cost of the partial route taken so far.
- $h(n)$ = admissible lower-bound estimate of remaining cost.
- Always expand the state with smallest f ; guaranteed to find the optimum.

State representation

A state is a pair (v, S) : current location v , set of *unvisited* centres S . Start: $(W, \{A, B, C, D\})$. Goal: $(W, \{\})$.

Admissible heuristic

For state (v, S) , define:

$$h(v, S) = \min_{u \in S \cup \{W\}} \text{cost}(v, u)$$

This is the cheapest single edge the vehicle must traverse next. It never overestimates, so it is admissible.

Step-by-Step Solution

Step 1 — Level-0 expansion (from W)

State	g	h	$f = g + h$
$(A, \{B, C, D\})$	10	$\min(15, 9, 14) = 9$	19
$(B, \{A, C, D\})$	18	$\min(15, 16, 8) = 8$	26
$(C, \{A, B, D\})$	12	$\min(9, 16, 11) = 9$	21
$(D, \{A, B, C\})$	20	$\min(14, 8, 11) = 8$	28

A* expands the lowest f : $(A, \{B, C, D\})$, $f = 19$.

Step 2 — Expand $(A, \{B, C, D\})$, $g = 10$

State	g	h	f
$(B, \{C, D\})$	$10 + 15 = 25$	$\min(16, 8) = 8$	33
$(C, \{B, D\})$	$10 + 9 = 19$	$\min(16, 11) = 11$	30
$(D, \{B, C\})$	$10 + 14 = 24$	$\min(8, 11) = 8$	32

Open list (sorted by f): $(C, \{A, B, D\})$ 21, $(B, \{A, C, D\})$ 26, $(D, \{A, B, C\})$ 28, $(C, \{B, D\})$ 30, $(D, \{B, C\})$ 32, $(B, \{C, D\})$ 33.

A* expands $(C, \{A, B, D\})$, $f = 21$.

Step 3 — Expand $(C, \{A, B, D\})$, $g = 12$

State	g	h	f
$(A, \{B, D\})$	$12 + 9 = 21$	$\min(15, 14) = 14$	35
$(B, \{A, D\})$	$12 + 16 = 28$	$\min(15, 8) = 8$	36
$(D, \{A, B\})$	$12 + 11 = 23$	$\min(14, 8) = 8$	31

Step 4 — Continue A* trace and enumerate key complete routes

Rather than trace every step, we identify the complete routes A* evaluates. There are $4! = 24$ tours; A* evaluates only those whose f -values stay below the best complete route found.

Route	Total Cost
$W \rightarrow A \rightarrow B \rightarrow D \rightarrow C \rightarrow W$	$10 + 15 + 8 + 11 + 12 = 56$
$W \rightarrow A \rightarrow C \rightarrow D \rightarrow B \rightarrow W$	$10 + 9 + 11 + 8 + 18 = 56$
$W \rightarrow C \rightarrow D \rightarrow B \rightarrow A \rightarrow W$	$12 + 11 + 8 + 15 + 10 = 56$
$W \rightarrow B \rightarrow D \rightarrow C \rightarrow A \rightarrow W$	$18 + 8 + 11 + 9 + 10 = 56$
$W \rightarrow A \rightarrow D \rightarrow B \rightarrow C \rightarrow W$	$10 + 14 + 8 + 16 + 12 = 60$
$W \rightarrow C \rightarrow A \rightarrow D \rightarrow B \rightarrow W$	$12 + 9 + 14 + 8 + 18 = 61$
$W \rightarrow A \rightarrow C \rightarrow B \rightarrow D \rightarrow W$	$10 + 9 + 16 + 8 + 20 = 63$
$W \rightarrow C \rightarrow A \rightarrow B \rightarrow D \rightarrow W$	$12 + 9 + 15 + 8 + 20 = 64$

Step 5 — Optimal solution and justification

Optimal route cost = 56. Four routes share this cost, all exploiting the two cheap edges:

- **B–D: cost 8** (the cheapest edge in the graph).
- **C–D: cost 11** and **A–C: cost 9**.

One optimal route: $W \rightarrow A \rightarrow B \rightarrow D \rightarrow C \rightarrow W = 10 + 15 + 8 + 11 + 12 = 56$.

A* guarantees optimality because it never returns a complete solution until all partial paths with $f < 56$ have been explored and shown to be worse or equal.

Step 6 — Why greedy fails

Greedy always picks the cheapest next edge. Starting at W:

1. $W \rightarrow A$ (cheapest from W = 10) ✓
2. $A \rightarrow C$ (cheapest unvisited from A = 9) ✓
3. $C \rightarrow D$ (cheapest unvisited from C = 11) ✓
4. $D \rightarrow B$ (cheapest unvisited from D = 8) ✓
5. $B \rightarrow W$ (only return = 18)
6. Total = $10 + 9 + 11 + 8 + 18 = 56$ ✓ (tied optimal in this case)

Greedy *happens* to hit optimal here, but this is coincidental. On different graphs, greedy commits to a cheap first step that forces expensive later steps, with no mechanism to recover. A* keeps all partial routes alive and ranked by f , guaranteeing it never misses the global optimum.

Question 3 — A* Algorithm: Expanding Node N

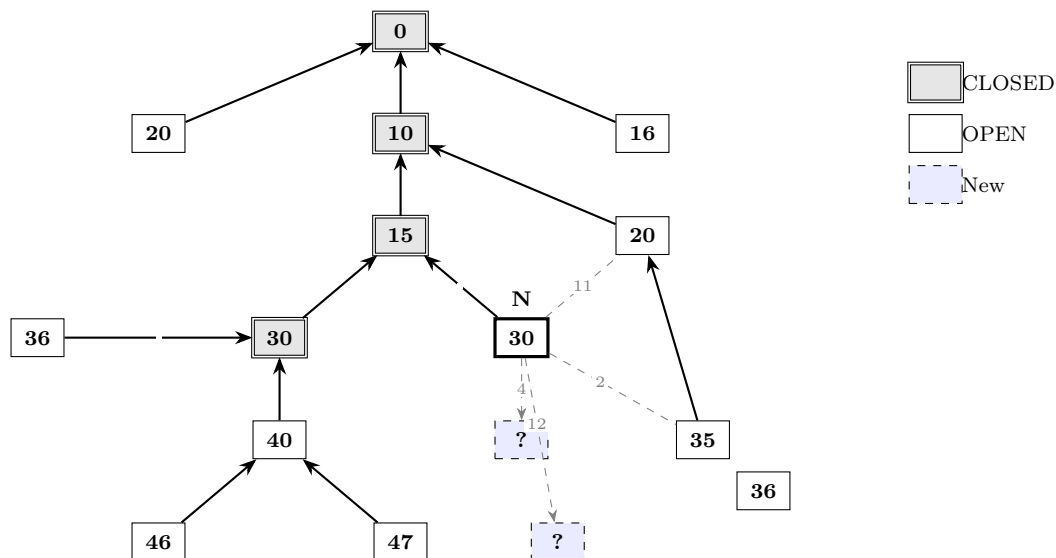
The Question

The figure below shows a graph being explored by A*. Node N (g -value = 30) is **about to be expanded**. After expansion, redraw the graph showing all changes.

Legend:

- **Double-bordered box** = node on CLOSED (already expanded).
- **Single-bordered box** = node on OPEN (discovered, awaiting expansion).
- **Dashed box** = node about to be added ("New").
- **Arrows** = back-pointers (direction: child $\rightarrow$ parent).
- **Edge labels** = step costs.

Graph BEFORE Expanding N



Reading the graph: Node N is the OPEN node with $g = 30$ (level 3, centre). It has **four** edges leading outward:

- Edge cost **4** to a *New* node (not yet in OPEN/CLOSED) — will give $g = 30 + 4 = 34$.
- Edge cost **12** to another *New* node — will give $g = 30 + 12 = 42$.
- Edge cost **11** to the existing OPEN node with $g = 20$ (right, level 2) — would give $g = 30 + 11 = 41 > 20$. *No update*.
- Edge cost **2** to the existing OPEN node with $g = 35$ (level 4) — would give $g = 30 + 2 = 32 < 35$. *Update: redirect back-pointer, set $g = 32$.*

Background Concepts

A* expansion procedure for node N

1. **Remove N from OPEN; add N to CLOSED** (visually: single box $\rightarrow$ double/shaded box).
2. **For each successor S of N :**
 - (a) Compute $g_{\text{new}} = g(N) + \text{cost}(N, S)$.
 - (b) If S is unknown: add to OPEN with $g(S) = g_{\text{new}}$, back-pointer to N .
 - (c) If $S \in \text{OPEN}$ and $g_{\text{new}} < g(S)$: update $g(S)$, redirect back-pointer to N .
 - (d) If $S \in \text{CLOSED}$ and $g_{\text{new}} < g(S)$: reopen S , update g , redirect pointer.
 - (e) Otherwise: discard (not an improvement).

Step-by-Step Solution

Step 1 — Apply expansion rules to each successor of N

Successor	Edge cost	g_{new}	Current g	Action
New node 1	4	$30 + 4 = 34$	— (unknown)	Add to OPEN with $g = 34$; back-pointer $\rightarrow N$.
New node 2	12	$30 + 12 = 42$	— (unknown)	Add to OPEN with $g = 42$; back-pointer $\rightarrow N$.
Node ($g = 20$)	11	$30 + 11 = 41$	20 (OPEN)	$41 > 20$: <i>no update</i> .
Node ($g = 35$)	2	$30 + 2 = 32$	35 (OPEN)	$32 < 35$: update g to 32, redirect back-pointer to N .

Step 2 — OPEN list after expansion of N

Before: OPEN contained $\{20_L, 16, 20_R, 36_L, 30_N, 35, 40, 36_R, 46, 47\}$.

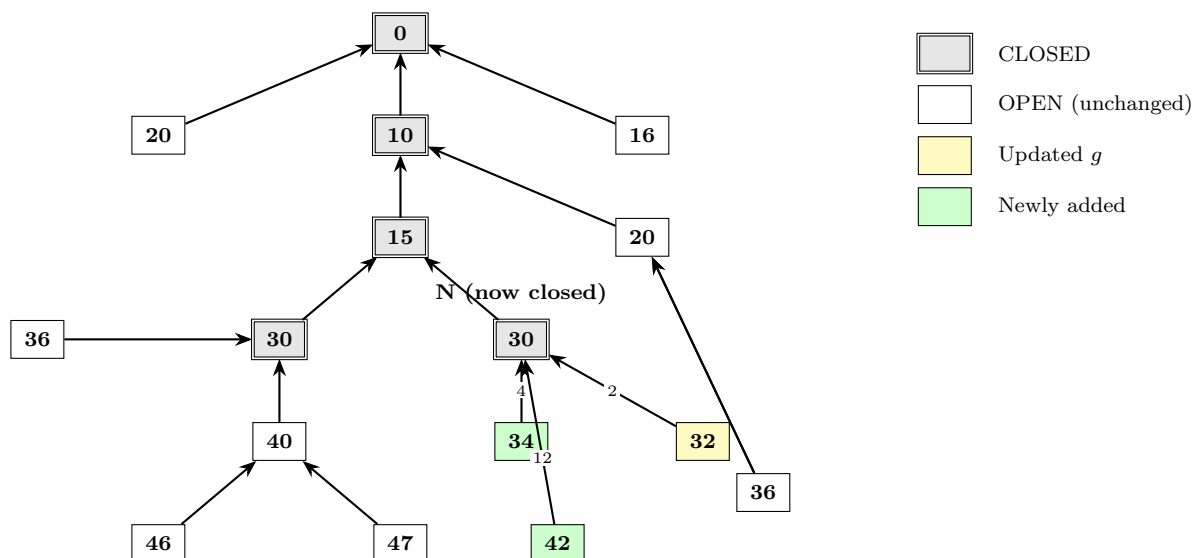
After:

- $N = 30$ moves to CLOSED.
- New node ($g=34$) joins OPEN.
- New node ($g=42$) joins OPEN.
- Node formerly $g=35$ is updated to $g=32$; back-pointer redirected to N .

OPEN = $\{16, 20_L, 20_R, 32^*, 34_{\text{new}}, 36_L, 36_R, 40, 42_{\text{new}}, 46, 47\}$

(sorted by g ; 32^* denotes the updated node).

Step 3 — Redrawn graph AFTER expanding N



Step 4 — Summary of changes

Node	Before	After
N ($g=30$, centre)	OPEN (single box)	CLOSED (double/shaded box)
New node 1	Not in graph	OPEN, $g = 34$, back-pointer $\rightarrow N$
New node 2	Not in graph	OPEN, $g = 42$, back-pointer $\rightarrow N$
Node $g = 35$	OPEN, $g=35$, ptr $\rightarrow$ old parent	OPEN, $g = 32$, ptr redirected $\rightarrow N$
Node $g = 20$ (right)	OPEN, $g=20$	Unchanged ($41 > 20$)
All other nodes	Unchanged	Unchanged

Key Insight: The key A* rule in action here: when a better path to an already-discovered node is found, **update** its g -value and redirect its back-pointer. The node stays on OPEN (don't remove it) — it just gets a cheaper label. If it had been on CLOSED, we would reopen it (with consistent/admissible heuristics this almost never happens).

Question 4 — Minimax with Alpha-Beta Pruning on Tic-Tac-Toe

The Question

Consider the Tic-Tac-Toe board below. X is MAX, O is MIN, and it is X's turn.

	C1	C2	C3
R1	X	O	
R2		X	O
R3			

X considers moves in order: R1C3, R2C1, R3C1, R3C3. O considers its responses in row-major order. Search depth = 2 plies (X moves, then O responds).

Evaluation function at horizon states:

$$E(s) = 100(W_X - W_O) + 10(T_X - T_O)$$

- W_X, W_O = number of *complete winning lines* for X, O.
- T_X, T_O = number of *open two-in-a-rows*: lines with exactly 2 marks of one player and 1 empty cell (the line must not contain the opponent's mark).

Task: Draw the minimax game tree (2 plies), compute $E(s)$ at all leaf nodes, apply alpha-beta pruning, and report X's optimal move.

Background Concepts

Minimax

MAX player (X) picks the move leading to the highest value; MIN player (O) picks the lowest. At leaf nodes, evaluate with $E(s)$. Back-propagate: MAX nodes take the maximum of children; MIN nodes take the minimum.

Alpha-Beta Pruning

Avoids evaluating branches that cannot change the final outcome.

- α = best value MAX can guarantee at or above current node. Init: $-\infty$.
- β = best value MIN can guarantee at or above current node. Init: $+\infty$.
- **β -cutoff (at MIN node):** if we find a child value $v \leq \alpha$, prune remaining children (MAX won't choose this branch).
- **α -cutoff (at MAX node):** if we find a child value $v \geq \beta$, prune remaining children (MIN won't allow this branch).

The 8 Tic-Tac-Toe lines

Rows: R1=(R1C1,R1C2,R1C3), R2=(R2C1,R2C2,R2C3), R3=(R3C1,R3C2,R3C3)
 Cols: C1=(R1C1,R2C1,R3C1), C2=(R1C2,R2C2,R3C2), C3=(R1C3,R2C3,R3C3)
 Diag: D1=(R1C1,R2C2,R3C3), D2=(R1C3,R2C2,R3C1)

Step-by-Step Solution

Step 1 — Analyse the initial board

<i>X</i>	<i>O</i>	-
-	<i>X</i>	<i>O</i>
-	-	-

Line	Contents	$T_X?$	$T_O?$
R1	X, O, -	No (blocked by O)	No (blocked by X)
R2	-, X, O	No (blocked by O)	No (blocked by X)
R3	-, -, -	No (0 X's)	No (0 O's)
C1	X, -, -	No (only 1 X)	No
C2	O, X, -	No (blocked by O)	No (blocked by X)
C3	-, O, -	No (only 1 O)	No (only 1 O)
D1	X, X, -	Yes (2X + empty)	No
D2	-, X, -	No (1 X)	No

Initial: $T_X = 1$ (D1), $T_O = 0$, $W_X = W_O = 0$.

Step 2 — X's move R1C3 (ply 1)

Board after X plays R1C3:

<i>X</i>	<i>O</i>	<i>X</i>
-	<i>X</i>	<i>O</i>
-	-	-

$W_X = 0$. D1=(X,X,-) ✓; D2=(X,X,-) ✓ $\rightarrow T_X = 2$. No O two-in-a-rows: $T_O = 0$.

Before O responds, base score = $10(2 - 0) = +20$.

O's legal moves (row-major): R2C1, R3C1, R3C2, R3C3.

O plays	Key analysis	$E(s)$
R2C1	Board: R2=(O,X,O) blocked. D1 (X,X,-)✓; D2 (X,X,-)✓. $T_X = 2, T_O = 0$.	+20
R3C1	D2=(X,X,O) blocked. C1=(X,-,O) only 1O. D1=(X,X,-)✓. $T_X = 1, T_O = 0$.	+10
R3C2	D1 still open; D2 still open. C2=(O,X,O) blocked. $T_X = 2, T_O = 0$.	+20
R3C3	D1=(X,X,O) blocked. D2 still open. C3=(-, O, O)→ $T_O = 1, T_X = 1, T_O = 1$.	0

MIN (O) picks the *minimum*: O plays R3C3 to get $E = 0$.

Value backed up for X's R1C3 = 0. Root MAX α updated: $\alpha = 0$.

Step 3 — X's move R2C1 (ply 1)

Board after X plays R2C1:

X	O	-
X	X	O
-	-	-

$W_X = 0$. C1=(X,X,-)✓; D1=(X,X,-)✓. $T_X = 2, T_O = 0$. Base = +20.

O's legal moves: R1C3, R3C1, R3C2, R3C3.

O plays	Key analysis	$E(s)$
R1C3	C3=(-, O, -) only 1O. R1=(X,O,O)→ $T_O = 1$. C1=(X,X,-)✓. $T_X = 1, T_O = 1$.	0

O's first response gives $E = 0 \leq \alpha = 0$: **β -cutoff** — **prune remaining O responses.**

Value for R2C1 ≤ 0 . MAX keeps $\alpha = 0$.

Step 4 — X's move R3C1 (ply 1)

Board after X plays R3C1:

X	O	-
-	X	O
X	-	-

$W_X = 0$. D1=(X,X,-)✓; C1=(X,-,X)✓. $T_X = 2, T_O = 0$. Base = +20.

O's legal moves: R1C3, R2C1, R3C2, R3C3.

O plays	Key analysis	$E(s)$
R1C3	C3=($_,$ O, $_$) only 1O; 0 wait: R1=(X,O,O) $\rightarrow T_O =$ 1. D2=(O,X, $_$) blocked. C1=(X, $_,$ X)✓. $T_X =$ 1, $T_O = 1$.	

O's first response gives $E = 0 \leq \alpha = 0$: **β -cutoff** — **prune remaining O responses.**

Value for R3C1 ≤ 0 . MAX keeps $\alpha = 0$.

Step 5 — X's move R3C3 (ply 1) — X WINS

Board after X plays R3C3:

X	O	-
-	X	O
-	-	X

D1 = (R1C1=X, R2C2=X, R3C3=X) — complete! $W_X = 1$.

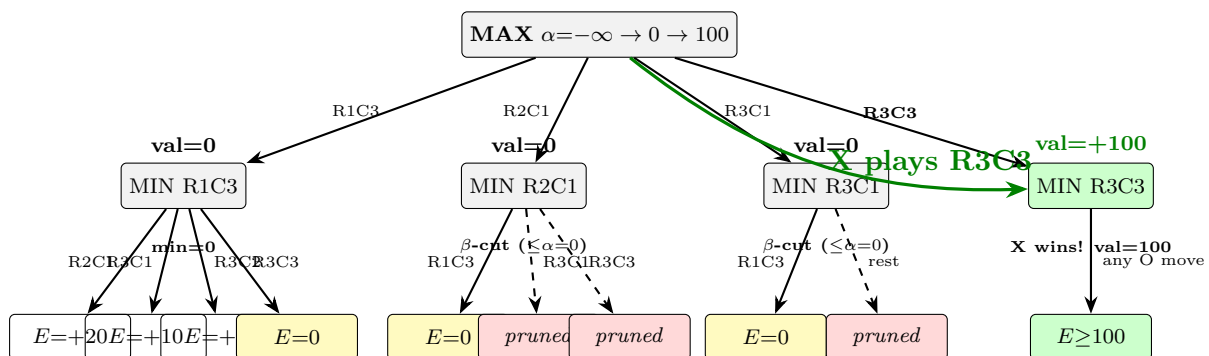
All leaves after O responds will have $W_X = 1$ (O cannot undo X's winning diagonal). With $W_X = 1, W_O = 0$:

$E(s) \geq 100(1 - 0) = 100$ (plus T terms, all positive for X).

MIN (O) picks the minimum over O's responses — but all are ≥ 100 . Backed-up value for R3C3 $\geq +100$.

Since $100 > \alpha = 0$, MAX updates: $\alpha = 100$. **X selects R3C3.**

Step 6 — Minimax tree with alpha-beta



Step 7 — Final answer

X's Move	Minimax Value	Remarks
R1C3	0	O counters with R3C3, yields $E = 0$
R2C1	0	O's first response already gives 0; pruned
R3C1	0	O's first response gives 0; pruned
R3C3	$\approx +100$	X wins on diagonal D1 immediately

X must play R3C3. This completes the main diagonal (R1C1, R2C2, R3C3), giving X an immediate win ($W_X = 1$). No O response can undo this, so every leaf scores ≥ 100 .

Alpha-beta summary:

- Root MAX: α begins at $-\infty$, rises to 0 after R1C3, stays 0 after R2C1, stays 0 after R3C1, jumps to +100 after R3C3.
- The β -cutoffs after R2C1 and R3C1 saved evaluating 6 leaf nodes that couldn't improve MAX's guaranteed value.

Question 5 — Intelligent Agent for Smart Farm Management

The Question

A large autonomous agricultural management system governs a smart farm with multiple crop zones, irrigation networks, weather stations, soil sensors, drones, and robotic equipment. It must monitor crop and soil conditions, predict irrigation needs, detect pests, coordinate machines, and take corrective action. Sensors are noisy; weather changes unexpectedly; zone actions affect other zones; objectives (yield, water, energy, disease control) compete.

Task: Formulate an appropriate intelligent-agent model. Give the PEAS description; classify the environment; analyse how a rational agent acts under incomplete/uncertain/conflicting information; propose and justify an agent structure; explain why locally beneficial actions may hurt long-term performance.

Background Concepts

PEAS

Every intelligent agent task is described by four components:

- **Performance measure** — what is success?
- **Environment** — what does the agent interact with?
- **Actuators** — how does it act?
- **Sensors** — how does it perceive?

Environment dimensions

- **Partially / Fully observable:** can the agent see the complete state?
- **Single / Multi-agent:** one decision-maker or several interacting?
- **Deterministic / Stochastic:** do actions have predictable outcomes?
- **Episodic / Sequential:** are decisions independent or do they affect future choices?
- **Static / Dynamic:** does the environment change while the agent deliberates?
- **Discrete / Continuous:** finite vs. infinite states and actions.
- **Known / Unknown:** does the agent know how the environment works?

Agent architectures (summary)

Simple reflex $\subset$ Model-based reflex $\subset$ Goal-based $\subset$ Utility-based $\subset$ Learning agent. Each level adds capability needed to handle more complex environments.

Step-by-Step Solution

Step 1 — PEAS Description

P — Performance	Crop yield per season; water efficiency (yield per litre); energy consumption; disease-to-containment response time; inter-zone balance; long-term soil health.
E — Environment	Physical farm: crop zones, soil layers, irrigation pipes, weather patterns, pest populations, plant diseases. Digital layer: sensor network, drone communication, scheduling queues.
A — Actuators	Irrigation valves; pesticide drones; robotic harvesters; fertiliser dispensers; ventilation/shade systems; alerts to human operators; inter-machine communication channels.
S — Sensors	Soil moisture probes; temperature/humidity sensors; weather stations; plant-health cameras (RGB/infrared); pest traps with counters; soil nutrient probes; drone GPS; water flow meters.

Step 2 — Environment Classification

Dimension	Classification	Justification
Observability	Partially observable	Sensors are noisy; root health is invisible; weather prediction is probabilistic; some zones may lack coverage.
Agents	Multi-agent	Drones, harvesters, and the central system all act autonomously.
Outcomes	Stochastic	Pest spread is probabilistic; irrigation effects depend on soil variability; weather is unpredictable.
Time	Sequential	Today's irrigation changes tomorrow's moisture; today's pesticide affects next season's soil.
Change	Dynamic	Crops grow, pests spread, weather evolves — all while the agent deliberates.
State space	Continuous	Soil moisture, temperature, nutrient levels, growth rates are real-valued.
Knowledge	Partially known	General agronomy is known; this farm's micro-climate behaviour must be learnt.

Step 3 — Rational Action Under Uncertainty

When sensor data is noisy, incomplete, or contradictory, a rational agent:

1. **Maintains a belief state.** Uses a probabilistic model (Bayesian network, Kalman filter) to estimate the true state from imperfect readings. Acts on a distribution of possible states, not a single observation.
2. **Fuses conflicting readings.** When sensors disagree, applies weighted averaging (by reliability), cross-validation with historical data, or majority voting. Anomalous readings are flagged for human review rather than acted on blindly.
3. **Applies the precautionary principle.** Under uncertainty, errs on the side of the less-risky action (e.g., slight under-irrigation to avoid waterlogging) unless the cost of inaction (pest outbreak) outweighs the risk.
4. **Discounts the future.** Uses a utility function with temporal discounting: actions that maximise immediate yield but deplete soil are penalised by the negative future utility they incur.
5. **Coordinates with other agents.** Uses a central scheduler or distributed negotiation protocol to avoid conflicting simultaneous actions (e.g., drone spraying while a harvester is in the same zone).

Step 4 — Proposed Agent Structure

The environment is partially observable, stochastic, sequential, dynamic, continuous, multi-agent, and partially known. The required agent structure is a **Learning Utility-Based Agent with an Internal World Model**.

Component	Function
Perception	Raw sensor data arrives; noise is filtered; readings are time-stamped and zone-tagged.
Internal belief model	Probabilistic map of the farm: moisture estimates with confidence intervals, pest probability gradients, crop health scores per zone, weather forecast. Updated with every new reading (Bayesian update).
Utility function	$U = w_1(\text{yield}) - w_2(\text{water}) - w_3(\text{energy}) - w_4(\text{disease loss}) + w_5(\text{soil health})$. Weights w_i encode the trade-offs.
Decision-making	Generates candidate action sequences; evaluates each by expected utility over the belief state; selects the sequence with highest expected utility.
Learning	Compares predicted outcomes to actual outcomes after each action; updates the belief model and utility weights. Over seasons, learns farm-specific patterns.
Actuation	Executes selected actions; communicates schedule to other autonomous machines to prevent conflicts.

Step 5 — Why Environment Properties Determine the Agent Choice

- **Partial observability → need internal belief model.** A simple reflex agent acts on current perception alone. Since sensors are noisy and incomplete, perceptions do not fully reveal the true state. The agent must maintain an internal model and update it probabilistically — ruling out simple reflex agents.
- **Stochastic + dynamic → need probabilistic reasoning.** A deterministic rule (“if moisture < 30% then irrigate”) fails when readings are wrong and irrigation effects depend on unpredictable weather. The agent must reason over distributions of outcomes.
- **Sequential + long horizon → need utility-based planning.** A locally optimal action (maximum pesticide for a small outbreak) may harm soil health, kill beneficial insects, and reduce next-season yield. A utility function with temporal discount forces the agent to weigh long-term consequences, which goal-based and reflex agents cannot do.
- **Multi-agent → need coordination.** Independent drones and harvesters may schedule conflicting actions. The central agent must enforce a global action schedule or participate in negotiation, ensuring cooperative rather than conflicting behaviour.
- **Partially known → need learning.** Textbook agronomy provides priors, but this specific farm’s soil, micro-climate, and pest dynamics must be discovered through observation. A learning component improves predictions over time, enabling increasingly rational decisions.

The local-vs-global optimality problem in practice:

Suppose zone 3 has a small pest outbreak. The *locally optimal* immediate action: spray pesticide aggressively to eliminate pests quickly (+50 on the disease-control metric).

However, the global utility is actually *lower* because:

- Excessive pesticide kills beneficial insects in zones 2 and 4 (−30 yield over next season).
- Pesticide residue degrades soil quality in zone 3 (−20 soil health, −15 yield next cycle).
- Water used in application reduces irrigation availability in zone 5 (−10 yield this season).

Net global utility: $+50 - 30 - 20 - 15 - 10 = -25$.

A **utility-based agent with a world model** evaluates the full impact and may instead choose a targeted low-dose drone spray, balancing containment speed against collateral damage. This is impossible for reflex agents (no world model) or goal-based agents (single goal, no trade-off).